

ESC112 Tutorial 8

1 Introduction

Topics to be covered: doubly linked list. Codes etc can be found in hello IITK.

2 Doubly linked list

In linked list, there is some data and a pointer to the next node. In a doubly linked list there are two pointers – one to the next and another to the previous. Accordingly, we can maintain the head as before, as well as a tail to access the list from both endpoints. As a result, one can travel in two directions from any node. This leads to efficient behavior in many cases. However, inserting and deleting becomes more complicated – we need to update the two pointers properly. A DLL (doubly linked list) structure can be defined as follows.

```
1 struct node{
2     int id;
3     char *name;
4     struct node *next;
5     struct node *prev;
6 };
7
8 typedef struct node Node;
9 typedef Node * List;
```

A doubly linked list looks something like: NULL $\leftarrow$ 10 $\leftrightarrow$ 20 $\leftrightarrow$ 30 $\rightarrow$ NULL. the left arrows are prev pointers and the right arrows are next pointers. Here, 10 is the head having NULL prev and 30 is the tail having NULL next. Note how easy it is now to print forward as well as reverse, which is cumbersome to do in singly linked list.

3 Example: music playlist

We will now see how to use DLL to maintain a playlist in a media player application.

3.1 Problem statement

```
1 Problem: Music Playlist Navigator using Doubly Linked List
2
3 You are required to implement a music playlist system using a doubly
  linked list.
4
5 Each song in the playlist is represented as a node containing:
6 - Song ID (integer)
7 - Song name (string without spaces)
8
9 The playlist supports navigation in both directions (next and previous),
  which must be efficiently handled using a doubly linked list.
```

```

1 Operations:
2
3 1. ADD_FRONT id name
4     Insert a new song at the beginning of the playlist.
5
6 2. ADD_END id name
7     Insert a new song at the end of the playlist.
8
9 3. ADD_AFTER id_new name_new id_existing
10    Insert a new song after the first occurrence of the song with
    id_existing.
11    If id_existing is not found, print "NOT_FOUND".
12
13 4. REMOVE id
14    Remove the first occurrence of the song with the given id.
15    If the song is not found, print "NOT_FOUND".
16
17 5. NEXT
18    Move the current pointer to the next song and print its details.
19    If playlist is empty, print "EMPTY".
20    If already at the end:
21        - If circular mode is OFF: print "NO_NEXT"
22        - If circular mode is ON: wrap to the first song and print it
23
24 6. PREV
25    Move the current pointer to the previous song and print its details.
26    If playlist is empty, print "EMPTY".
27    If already at the beginning:
28        - If circular mode is OFF: print "NO_PREV"
29        - If circular mode is ON: wrap to the last song and print it
30
31 7. PLAY id
32    Move the current pointer to the song with the given id and print it.
33    Output format is same as CURRENT.
34    If not found, print "NOT_FOUND".
35
36 8. CURRENT
37    Print the current song.
38    If playlist is empty, print "EMPTY".
39
40 9. PRINT_FORWARD
41    Print all songs from head to tail.
42
43 10. PRINT_BACKWARD
44    Print all songs from tail to head.
45
46 11. TOGGLE_LOOP
47    Toggle circular mode ON/OFF.
48    In circular mode, navigation wraps logically; modifying actual links
    is optional.

```

```
1 Rules:
2
3 - Initially, the playlist is empty.
4 - The current pointer is NULL when the playlist is empty.
5 - When the first song is added, it becomes the current song.
6 - Song IDs are unique.
7 - Unless the list is empty, it must have a current song.
8
9 - If the current song is deleted:
10     - Prefer next node if available
11     - Otherwise move to previous node
12     - If the deleted node was the only node, current becomes NULL
13
14 - PRINT_FORWARD and PRINT_BACKWARD must always terminate at NULL and
    should not loop infinitely, even in circular mode.
```

```

1 Input Format:
2
3 - Each line contains one operation.
4 - Input is given until EOF.
5
6 -----
7
8 Output Format:
9
10 - For NEXT, PREV, PLAY, and CURRENT:
11     id:name
12
13 - For PRINT_FORWARD and PRINT_BACKWARD:
14     id:name <-> id:name <-> ...
15
16 - For error cases, print:
17     NOT FOUND
18     NO NEXT
19     NO PREV
20     EMPTY
21
22 -----
23
24 Constraints:
25
26 1 <= total number of operations <= 1000
27 1 <= id <= 10^6
28 Song name length <= 20
29
30 -----
31
32 Sample Input:
33
34 ADD_END 1 songA
35 ADD_END 2 songB
36 ADD_END 3 songC
37 CURRENT
38 NEXT
39 NEXT
40 NEXT
41 PREV
42 REMOVE 2
43 PRINT_FORWARD
44
45 -----
46
47 Sample Output:
48
49 1:songA
50 2:songB
51 3:songC
52 NO NEXT
53 2:songB
54 1:songA <-> 3:songC

```

3.2 Solution code

One of the main sources of error in a linked list implementation is accessing `l -> member` when `l` is `NULL` or freed – this causes runtime error. Avoiding this requires a careful case by case analysis. An textbook example is given below for freeing a list.

```
1 for(List p = head; p != NULL; p = p -> next) /* WRONG */
2     free(p);
```

The right way is to save whatever is needed before freeing:

```
1 for(List p = head; p != NULL; p = q) {
2     q = p->next;
3     free(p);
4 }
```

We will only discuss the main function, an add function, and possibly the remove function if time permits. Basically, three node pointers head, tail, and current will be maintained. These will be implicitly accessed and possibly updated by the functions. We need to take care to properly update the next and prev pointers in case of add or remove. The header looks as follows.

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4
5 #define LINEMAX 100
6 #define SONGNAMEMAX 20
7 #define COMMANDMAX 20
8
9 struct node{
10     int id;
11     char *name;
12     struct node *next;
13     struct node *prev;
14 };
15
16 typedef struct node Node;
17 typedef Node * List;
```

`sscanf` is a new function for us. It is like `scanf` except it reads from a *string* given in the first argument. as always, these `scanf` family of functions returns us how many `%` arguments was successfully read. Using `fgets` to read a line and doing `sscanf` on that line is a standard idiom to read a variety of commands with a varying number of arguments.

```

1 int main(){
2     char line[LINEMAX];
3     char command[COMMANDMAX];
4
5     int n;
6     int id;
7     char name[SONGNAMEMAX];
8     int id_existing;
9
10    List head = NULL;
11    List tail = NULL;
12    List current = NULL;
13
14    int circ = 0;
15
16    while(fgets(line, LINEMAX, stdin) != NULL){
17        n = sscanf(line, "%s_%d_%s_%d", command, &id, name, &id_existing);
18        if(strcmp(command, "ADD_FRONT") == 0 && n == 3){
19            add_front(id, name, &head, &tail, &current);
20        }
21        else if(strcmp(command, "ADD_END") == 0 && n == 3){
22            add_end(id, name, &head, &tail, &current);
23        }
24        else if(strcmp(command, "ADD_AFTER") == 0 && n == 4){
25            add_after(id, name, id_existing, &head, &tail, &current);
26        }
27        else if(strcmp(command, "REMOVE") == 0 && n == 2){
28            remove_node(id, &head, &tail, &current);
29        }
30        else if(strcmp(command, "NEXT") == 0 && n == 1){
31            next(circ, &head, &tail, &current);
32        }
33        else if(strcmp(command, "PREV") == 0 && n == 1){
34            prev(circ, &head, &tail, &current);
35        }
36        else if(strcmp(command, "PLAY") == 0 && n == 2){
37            play(id, &head, &tail, &current);
38        }
39        else if(strcmp(command, "CURRENT") == 0 && n == 1){
40            print_current(current);
41        }
42        else if(strcmp(command, "PRINT_FORWARD") == 0 && n == 1){
43            print_forward(head);
44        }
45        else if(strcmp(command, "PRINT_BACKWARD") == 0 && n == 1){
46            print_backward(tail);
47        }
48        else if(strcmp(command, "TOGGLE_LOOP") == 0 && n == 1){
49            circ = !circ;
50        }
51        else{
52            printf("invalid_use_of_command:_%s\n", command);
53            continue;
54        }
55    }
56
57    free_list(head);
58 }

```

```

1 void add_front(int id, char *name, List *head, List *tail, List *current){
2     //set up the new node to be put as head
3     List new = calloc(1, sizeof(struct node));
4     new -> id = id;
5     new -> name = strdup(name);
6     if(new ->name == NULL){
7         exit(1);
8     }
9     new -> prev = NULL;
10
11    //now put it as head
12    if(*head == NULL){
13        //means empty list
14        //tail and current also must be NULL
15        new -> next = NULL;
16        *head = *tail = *current = new;
17        return;
18    }
19    else{
20        new -> next = *head;
21        (*head) -> prev = new; //() needed
22        *head = new;
23        return; //*tail unchanged
24    }
25 }
26 }

```

The * is only needed when you need to reach back and update something that belongs to the caller.

Adding in the end is similar. Removing a node is given below. The rest of the code is in appendix and also in hello IITK.

```

1 void free_node(List l){
2     if(l -> name != NULL)
3         free(l -> name);
4     free(l);
5 }

```



```

1 void remove_node(int id, List *head, List *tail, List *current){
2     List l = *head;
3     for(; l != NULL && l -> id != id; l = l -> next)
4         ;
5     if(l == NULL){
6         printf("NOT_FOUND\n");
7         return;
8     }
9
10    //update current
11    if((*current) -> id == id){
12        if((*current) -> next != NULL)
13            *current = (*current) -> next;
14        else if((*current) -> prev != NULL)
15            *current = (*current) -> prev;
16        else{                                     #checks: current at start or end or mid!
17            //list was singleton
18            free_node(*current);
19            *current = *head = *tail = NULL;
20            return;
21        }
22    }
23
24    //now remove l
25    if(l -> next == NULL){
26        //tail will be removed
27        //non-singleton -- tail has prev
28        l -> prev -> next = NULL;
29        *tail = l -> prev;
30        free_node(l);
31        return;                                     #checks: l at start or end or mid
32    }
33    else if(l -> prev == NULL){
34        //head will be removed
35        //non-singleton -- head has next
36        l -> next -> prev = NULL;
37        *head = l -> next;
38        free_node(l);
39        return;
40    }
41    else{
42        //l has both prev and next
43        //head and tail do not change
44        l -> next -> prev = l -> prev;
45        l -> prev -> next = l -> next;
46        free_node(l);
47        return;
48    }
49 }

```

Appendix with rest of the code

```

1 void next(int circ, List *head, List *tail, List *current){
2     if(*current == NULL){
3         printf("EMPTY\n");
4         return;
5     }
6
7     List temp = (*current) -> next;
8     if(temp == NULL && circ == 0){
9         //current stays as it is
10        printf("NO_NEXT\n");
11        return;
12    }
13    else if(temp == NULL && circ == 1){
14        temp = *head;
15    }
16    *current = temp;
17
18    printf("%d:%s\n", temp -> id, temp -> name);
19 }
20
21 void prev(int circ, List *head, List *tail, List *current){
22     if(*current == NULL){
23         printf("EMPTY\n");
24         return;
25     }
26
27     List temp = (*current) -> prev;
28     if(temp == NULL && circ == 0){
29         printf("NO_PREV\n");
30         //current stays as it is
31         return;
32     }
33     else if(temp == NULL && circ == 1){
34         temp = *tail;
35     }
36     *current = temp;
37
38     printf("%d:%s\n", temp -> id, temp -> name);
39 }

```

u only use * with the pointers if they r being introduced from outside the locality of the function!!!!

```

1 void add_end(int id, char *name, List *head, List *tail, List *current){
2     //set up the new node to be put as tail
3     List new = calloc(1, sizeof(struct node));
4     new -> id = id;
5     new -> name = strdup(name);
6     if(new ->name == NULL){
7         exit(1);
8     }
9     new -> next = NULL;
10
11     //now put it as tail
12     if(*tail == NULL){
13         //empty list -- head and current also NULL
14         new -> prev = NULL;
15         *head = *tail = *current = new;
16         return;
17     }
18     else{
19         new -> prev = *tail;
20         (*tail) -> next = new;
21         *tail = new;
22         return;        // *head unchanged
23     }
24 }
25
26 void add_after(int id_new, char *name_new, int id_existing, List *head,
27 List *tail, List *current){
28     for(List l = *head; l != NULL; l = l -> next)
29         if(l -> id == id_existing){
30             //set up new node to be put
31             List new = calloc(1, sizeof(struct node));
32             new -> id = id_new;
33             new -> name = strdup(name_new);
34             if(new ->name == NULL){
35                 exit(1);
36             }
37
38             //now put it
39             new -> next = l -> next;
40             if(l -> next != NULL){
41                 l -> next -> prev = new;
42             }
43             else{
44                 //l was the tail
45                 *tail = new;
46             }
47             l -> next = new;
48             new -> prev = l;
49             return;
50         }
51     //add after can never change current
52     printf("NOT_FOUND\n");
53     return;
54 }

```

```

1 void play(int id, List *head, List *tail, List *current){
2     for(List l = *head; l != NULL; l = l -> next){
3         if(l -> id == id){
4             *current = l;
5             printf("%d:%s\n", l -> id, l -> name);
6             return;
7         }
8     }
9     printf("NOT_FOUND\n");
10 }
11
12 void print_current(List current){
13     if(current == NULL)
14         printf("EMPTY\n");
15     else
16         printf("%d:%s\n", current -> id, current -> name);
17 }
18
19 void print_forward(List head){
20     if(head == NULL){
21         printf("EMPTY\n");
22         return;
23     }
24     printf("%d:%s", head -> id, head -> name);
25
26     for(List l = head -> next; l != NULL; l = l -> next)
27         printf("<-> %d:%s", l -> id, l -> name);
28     printf("\n");
29 }
30
31 void print_backward(List tail){
32     if(tail == NULL){
33         printf("EMPTY\n");
34         return;
35     }
36     printf("%d:%s", tail -> id, tail -> name);
37
38     for(List l = tail -> prev; l != NULL; l = l -> prev)
39         printf("<-> %d:%s", l -> id, l -> name);
40     printf("\n");
41 }
42
43 void free_list(List head){
44     List l = head, temp;
45     while(l != NULL){
46         temp = l;
47         l = l -> next;
48         free_node(temp);
49     }
50 }

```